

RAPPORT DE PROJET FINAL : CLASSIFICATION D'IMAGES

*Comparaison Systématique entre Apprentissage Automatique Traditionnel (Transfer Learning)
et Réseaux de Neurones Convolutifs (CNN) Personnalisés*

Étudiant :	Abdrafith ZONGO
Module :	Réseaux de Neurones Artificiels (RNA)
Master :	M1 - Vision par Ordinateur et Intelligence Artificielle (VOIA)
Dépôt GitHub :	https://github.com/Abdrafith-ZONGO/classification-images-traditionnel-vs-cnn.git

INTRODUCTION

La classification d'images constitue l'un des piliers fondamentaux de la vision par ordinateur moderne. Ses applications couvrent des domaines critiques tels que l'imagerie médicale (détection de pathologies), l'observation de la Terre par satellites (suivi environnemental), et l'intelligence artificielle générale. Historiquement, la classification reposait sur des pipelines composés de deux phases distinctes : l'extraction de descripteurs visuels fabriqués à la main (comme SIFT ou HOG) suivie de la classification par des algorithmes classiques (SVM, k-NN, etc.). L'avènement du Deep Learning a complètement bouleversé ce paradigme en unifiant ces deux phases au sein d'une architecture unique : les Réseaux de Neurones Convolutifs (CNN), capables d'apprendre conjointement les représentations visuelles et le critère de décision.

Ce projet propose une évaluation critique et comparative de ces deux approches. Nous étudions et mettons en concurrence :

- **Pipeline 1 (Apprentissage Traditionnel par Transfer Learning) :** Nous exploitons des modèles convolutifs profonds de pointe pré-entraînés sur la base géante ImageNet (AlexNet, VGG16, et InceptionV3) en tant qu'extracteurs de caractéristiques universels (embeddings). Ces vecteurs de caractéristiques de haute dimension sont ensuite passés à des classifieurs classiques (Machine à Vecteurs de Support (SVM), k-Plus Proches Voisins (k-NN), Arbre de Décision, et Naïve Bayes).
- **Pipeline 2 (Apprentissage Profond direct) :** Nous concevons et entraînons complètement à partir de zéro (from scratch) un réseau de neurones convolutif personnalisé (CNNPerso) sur les images brutes sans extraction préalable.

L'analyse comparative est menée sur quatre jeux de données réels et variés pour éprouver la généralisation des modèles : Iris Flowers, COVID-19 X-Ray, Wildfire Satellite Images, et DTD (Describable Textures Dataset).

1. PROBLÉMATIQUE ET ENJEUX DE L'ÉTUDE

La comparaison entre le Transfer Learning et l'entraînement From Scratch soulève des questions technologiques et économiques cruciales :

- **Le compromis exactitude / volume de données :** Un réseau de neurones profond possède des millions de paramètres qui nécessitent en théorie des volumes de données conséquents pour converger sans surapprentissage. Le Transfer Learning permet-il de s'affranchir de cette contrainte sur de très petits datasets ?
- **L'empreinte et le coût de calcul (CPU vs GPU) :** L'apprentissage profond de bout en bout nécessite d'ajuster l'ensemble des poids du réseau par rétropropagation du gradient, un processus extrêmement lourd. Les méthodes traditionnelles sur features pré-extraites offrent-elles un

avantage décisif en temps de calcul, notamment pour des environnements contraints (utilisation CPU) ?

- **Taille de stockage et déploiement embarqué** : Quel est l'impact de la taille mémoire finale des modèles sur les possibilités d'intégration dans des systèmes embarqués (drones, smartphones) ?

2. PRÉSENTATION DES JEUX DE DONNÉES ET PRÉTRAITEMENTS

Nous analysons quatre bases de données aux caractéristiques structurelles distinctes :

- **Iris Flowers (421 images, 3 classes)** : Contient des images de fleurs d'Iris réparties en trois variétés (Setosa, Versicolor, Virginica). Ce dataset pose le défi du très faible volume de données et de la forte ressemblance entre les classes Versicolor et Virginica.
- **COVID-19 X-Ray (743 images, 2 classes)** : Images médicales de radiographies thoraciques réparties en deux classes (CT_COVID et CT_NonCOVID). Il s'agit d'un domaine où la précision de la classification est cruciale et les textures pulmonaires subtiles.
- **Wildfire Satellite Images (1 832 images, 2 classes)** : Images satellites de zones forestières classées selon la présence ou l'absence de feux (fire et nofire). C'est notre plus grand dataset, présentant de forts contrastes de couleur.
- **DTD (540 images, 9 classes)** : Échantillon de textures complexes d'animaux sauvages structuré en 9 classes (antelope, badger, butterfly, cat, chimpanzee, cow, dragonfly, eagle, elephant). Il représente un cas de figure très complexe : peu d'images par classe et un nombre de classes élevé.

Nettoyage et Prétraitements :

- Nettoyage automatique : Un filtrage programmatique rigoureux est effectué pour exclure les répertoires et fichiers parasites présents dans les dossiers sources (comme le dossier INF5082 présent dans le dossier Iris). De plus, chaque image est lue par PIL et validée par la fonction.verify () pour écarter les images corrompues.
- Split stratifié : Pour tous les jeux de données, les images sont séparées en 70% pour l'entraînement et 30% pour le test. La stratification garantit que la proportion de chaque classe est rigoureusement préservée dans les deux ensembles.
- Redimensionnement et Normalisation standard : Les images sont redimensionnées en 224x224 pour VGG16/AlexNet, 299x299 pour InceptionV3, et 128x128 pour notre CNN perso. Elles sont converties en tenseurs et normalisées selon les moyennes et écarts-types de référence d'ImageNet: mean=[0.485, 0.456, 0.406] et std=[0.229, 0.224, 0.225].
- Augmentation de données : Pour entraîner le CNN du Pipeline 2, nous appliquons de l'augmentation de données (retournement horizontal aléatoire, rotation jusqu'à 15 degrés, et

légère variation de luminosité/contraste via ColorJitter) sur l'ensemble d'entraînement pour améliorer la robustesse et éviter le surapprentissage.

3. DESCRIPTION ARCHITECTURALE DES MODÈLES ET EXTRACTEURS

Afin de justifier pleinement notre démarche, nous détaillons ici la structure des architectures convolutives exploitées.

3.1 Modèles pré-entraînés (Pipeline 1)

- VGG16 (138 millions de paramètres) : Développé par Oxford, ce modèle se caractérise par une architecture très homogène utilisant de petits filtres de convolution de 3x3 empilés en cascade (profondeur de 64, 128, 256, 512 filtres) séparés par des couches de Max Pooling. La force de VGG16 réside dans sa capacité à capter des représentations très riches au prix d'un poids mémoire conséquent (plus de 500 Mo sur le disque).
- AlexNet (61 millions de paramètres) : Réseau pionnier du Deep Learning (vainqueur d'ImageNet 2012), il utilise des filtres de convolution de grande taille lors des premières couches (11x11 puis 5x5) associés à des activations ReLU et du MaxPooling. Il représente une option plus légère mais possède néanmoins 61 millions de paramètres, majoritairement situés dans ses couches fully connected.
- InceptionV3 (27 millions de paramètres) : Conçu par Google, ce réseau introduit les modules "Inception". Au lieu d'empiler les convolutions de manière séquentielle, Inception applique des convolutions de tailles différentes (1x1, 3x3, 5x5) en parallèle à chaque étape et concatène leurs sorties. Cela lui permet de capter des motifs visuels à des échelles variées tout en divisant par 5 le nombre de paramètres par rapport à VGG16.

3.2 Réseau Convolutif Personnalisé CNNPerso (Pipeline 2)

Notre réseau a été dimensionné pour s'adapter à la résolution d'entrée de 128x128 pixels tout en limitant le risque d'overfitting. Il se compose de 4 blocs convolutionnels successifs suivis d'un classifieur dense (fully connected) :

- Bloc Convolutionnel 1 : Une couche de convolution bidimensionnelle (Conv2D) qui projette les 3 canaux d'entrée RGB vers 32 filtres de taille 3x3 (avec padding=1). Elle est immédiatement suivie d'une Batch Normalization pour stabiliser les gradients, d'une activation ReLU, et d'une couche de Max Pooling 2x2. Cette opération réduit la dimension spatiale de 128x128 à 64x64.


```

├── features/           : Cache des features extraites (.npy)
├── models/            : Poids (.pth) et classifieurs (.pkl)
├── reports/           : Matrices et graphiques de performance
├── main.py            : Script principal (orchestration CLI)
├── Projet_Final_Notebook.ipynb : Notebook interactif pas à pas
├── requirements.txt    : Bibliothèques Python requises
└── README.txt         : Instructions d'installation et usage

```

4.2 Fichier src/config.py (Configuration globale)

Ce script centralise l'ensemble des constantes, des répertoires d'entrée/sortie, et des hyperparamètres pour assurer la reproductibilité parfaite des expériences.

- SEED = 42 : Fixe la graine aléatoire pour numpy, torch et scikit-learn.
- DEVICE : Détermine automatiquement si une carte graphique compatible CUDA est disponible, sinon bascule sur le processeur (CPU).
- DATASET_PATHS : Dictionnaire contenant les chemins physiques absolus vers les dossiers de chaque dataset.
- DATASET_CLASSES : Dictionnaire définissant la liste exacte des classes valides par dataset.
- TAILLE_IMAGE_VGG_ALEX (224), TAILLE_IMAGE_INCEPTION (299), TAILLE_IMAGE_CNN (128) : Définit les résolutions de prétraitement d'images.
- FEATURES_DIR, MODELS_DIR, REPORTS_DIR : Chemins des répertoires de sauvegarde, créés automatiquement au chargement du module.
- BATCH_SIZE = 32, EPOCHS = 15, LEARNING_RATE = 0.001, PROPORTION_TEST = 0.3 : Réglage des hyperparamètres de base.

4.3 Fichier src/dataset.py (Gestion des données)

Ce fichier prend en charge la lecture physique des images, la vérification de leur intégrité, l'application des transformations PyTorch et la création des DataLoaders.

- Fonction obtenir_transforms(taille_image, augmentation=False) :
 - Arguments : taille_image (int), augmentation (bool).
 - Rôle : Retourne un objet transforms.Compose de PyTorch. Si augmentation est vrai, intègre des transformations géométriques (flips, rotations) et chromatiques. Sinon, effectue uniquement le redimensionnement et la normalisation avec les constantes ImageNet.
- Classe ImageDatasetCustom(Dataset) :

- Rôle : Héritage de la classe Dataset de PyTorch. Son constructeur prend en argument les listes de chemins d'images et de labels, ainsi que l'objet de transformation. La méthode `__getitem__(idx)` ouvre l'image, la force en format RGB, applique la transformation et retourne le tenseur avec son label numérique.

- Fonction `collecter_donnees_dataset(nom_dataset)` :

- Arguments : `nom_dataset` (str).

- Rôle : Parcourt les sous-dossiers autorisés. Vérifie que les fichiers ont des extensions valides (.png, .jpg, etc.) et utilise `PIL.Image.open().verify()` pour filtrer les images corrompues.

- Retourne : La liste des chemins d'images, la liste des labels correspondants, et la liste des noms de classes.

- Fonction `preparer_loaders(nom_dataset, taille_image, batch_size=32, test_size=0.3)` :

- Arguments : `nom_dataset` (str), `taille_image` (int), `batch_size` (int), `test_size` (float).

- Rôle : Réalise le split stratifié (via `train_test_split` de scikit-learn avec `stratify=labels`) pour préserver l'équilibre des classes dans le train et le test. Instancie les objets `ImageDatasetCustom` (avec augmentation pour le train) et retourne deux chargeurs `DataLoader`.

4.4 Fichier `src/extractors.py` (Extraction des caractéristiques)

Ce script convertit les modèles convolutifs pré-entraînés profonds en extracteurs de caractéristiques statiques (Transfer Learning).

- Fonction `charger_modele_preentraine(nom_modele)` :

- Arguments : `nom_modele` (str: "AlexNet", "VGG16" ou "InceptionV3").

- Rôle : Télécharge le modèle chargé avec ses poids d'origine (ImageNet). Identifie la dernière couche fully connected (la couche de classification finale) et la remplace par une couche `nn.Identity()`. Désactive les calculs de gradients.

- Retourne : Le modèle modifié et la dimension du vecteur de sortie (4096 pour AlexNet/VGG16, 2048 pour InceptionV3).

- Fonction `extraire_features(modele, loader)` :

- Arguments : `modele` (nn.Module), `loader` (DataLoader).

- Rôle : Parcourt les batchs du loader, pousse les tenseurs sur le DEVICE sélectionné, exécute le passage avant (forward pass) sans calcul de gradients (`torch.no_grad()`), puis aplatit et accumule les caractéristiques sous forme de vecteurs NumPy.

- Retourne : Un tableau numpy bidimensionnel X (features) et un tableau unidimensionnel y (labels).
- Fonction `obtenir_features_dataset(nom_dataset, nom_modele, forcer_reextraction=False)` :
- Arguments : `nom_dataset` (str), `nom_modele` (str), `forcer_reextraction` (bool).
- Rôle : Vérifie si les fichiers .npy d'entraînement et de test pour le couple dataset/modèle existent déjà dans features/. Si oui, les charge directement pour un gain de temps majeur. Sinon, déclenche le pipeline complet d'extraction et sauvegarde les tableaux numpy sur le disque.

4.5 Fichier `src/models_traditional.py` (Classifieurs classiques)

Ce fichier définit la grille de recherche des hyperparamètres et l'entraînement avec validation croisée des classifieurs classiques sur les features pré-extraites.

- Fonction `obtenir_grille_parametres(nom_classifieur)` :
- Arguments : `nom_classifieur` (str).
- Rôle : Renvoie les dictionnaires de recherche pour scikit-learn (SVM: C et kernels; k-NN: n_neighbors, weights; Decision Tree: max_depth; Naïve Bayes: var_smoothing).
- Fonction `instancier_classifieur_base(nom_classifieur)` :
- Rôle : Renvoie le modèle scikit-learn brut. Pour le SVM, paramètre `probability=True` afin d'autoriser le calcul des probabilités indispensables au tracé des courbes ROC.
- Fonction `entraîner_evaluer_classique(nom_dataset, nom_modele_extraction, nom_classifieur, X_train, y_train, X_test, y_test, optimiser=True)` :
- Rôle : Instancie et applique un standardiseur `StandardScaler` pour centrer et réduire les caractéristiques. Lance un `GridSearchCV` avec 3 splits (3-fold cross validation) en optimisant la métrique F1-score pondérée. Calcule les métriques de performance et mesure le temps d'entraînement pur.
- Sauvegarde : Enregistre le modèle final, le scaler associé, les hyperparamètres optimaux et les scores dans un fichier .pkl.

4.6 Fichier `src/models_cnn.py` (Réseau Convolutif Personnalisé)

Définit l'architecture de notre réseau convolutionnel et sa boucle d'apprentissage.

- Classe `CNNPerso(nn.Module)` :
- Architecture : Décrite dans la section 3.2.

- Fonction `entraîner_evaluer_cnn(...)` :

- Rôle : Boucle d'entraînement standard sur le nombre d'époques spécifié. Utilise la fonction de perte `CrossEntropyLoss` et l'optimiseur `Adam`. Évalue le modèle sur le jeu de test pour enregistrer l'historique et sauvegarde les poids finaux dans un fichier `.pth`.

4.7 Fichier `src/utils.py` (Visualisation et Graphiques)

- `tracer_courbes_apprentissage` : Trace l'évolution temporelle de la Loss et de l'Accuracy (Train vs Test) du CNN.

- `tracer_matrice_confusion` : Génère des diagrammes matriciels colorés de confusion avec `Seaborn`.

- `tracer_courbe_roc` : Génère les courbes ROC (cas binaire et approche One-vs-Rest en calculant la macro-moyenne des aires sous la courbe AUC).

- `tracer_comparaison_globale` : Dessine le diagramme général de synthèse des précisions par dataset et classement final.

4.8 Fichier `main.py` (Orchestrateur global)

Ce script orchestre les deux pipelines sur les datasets choisis, génère les graphiques d'évaluation et compile l'ensemble des métriques de performance dans un fichier CSV central `'reports/comparaison_globale.csv'`.

5. MÉTHODOLOGIE ET JUSTIFICATIONS TECHNIQUES

- Pourquoi normaliser avec `StandardScaler` ? Les algorithmes classiques (comme le SVM ou le k-NN) dépendent fortement du calcul de distances géométriques. Si certaines dimensions des features issues de VGG16 (dimension 4096) présentent des écarts ou des amplitudes disproportionnées, elles domineraient injustement le calcul. Le standardiseur recentre la distribution à une moyenne de 0 et un écart-type de 1.

- Pourquoi le noyau RBF pour le SVM ? Le noyau RBF (Radial Basis Function) permet de séparer les données projetées dans un espace non linéaire de dimension infinie. C'est l'approche idéale quand les classes ne sont pas séparables statistiquement dans l'espace d'origine.

- Pourquoi la validation croisée (`GridSearchCV`) ? Optimiser les hyperparamètres (k pour le k-NN, C et le noyau pour le SVM) directement sur l'ensemble de test biaise l'évaluation (data leakage). En effectuant une recherche sur grille validée par 3 sous-ensembles (3-fold cross validation) sur le train, nous garantissons l'intégrité de l'ensemble de test.

- Pourquoi concevoir le CNN ainsi (BatchNorm + Dropout) ? Les modèles de Deep Learning ont

tendance à surapprendre sur de petites bases. La Batch Normalisation à chaque bloc convolutif stabilise les activations intermédiaires, accélérant ainsi la convergence. Le Dropout (50%) force le réseau à ne pas dépendre d'une unique combinaison de neurones pour décider, améliorant ainsi la généralisation.

6. JEUX DE LIVRABLES ET ÉLÉMENTS DU DOSSIER DE SOUMISSION

Afin de répondre scrupuleusement aux critères d'évaluation, le dossier de soumission a été structuré sous la forme d'une archive compressée (Projet_Final_Classification.zip) contenant les éléments suivants :

- **Le Rapport PDF Complet (Rapport_Projet_Final.pdf)** : Ce document de synthèse universitaire, détaillé de A à Z, exporté en format standard PDF et mis en page en police classique Times New Roman.
- **Le Notebook Interactif de Démonstration (Projet_Final_Notebook.ipynb)** : Notebook Jupyter complet destiné à l'évaluation interactive. Il contient la détection matérielle, l'exploration et le filtrage des données (Iris), l'exécution isolée d'un pipeline ML, l'entraînement et l'affichage des courbes d'apprentissage de notre CNN sur 10 époques pour démonstration.
- **Les Scripts Python Sources (src/) et main.py** : Code source modulaire et documenté pour le nettoyage, l'extraction de features, l'entraînement classique et convolutionnel, et la génération des graphiques.
- **Les Modèles Entraînés Sauvegardés (models/)** : Modèles classiques optimisés (.pkl) et poids finaux de notre CNN (.pth) pour chaque dataset.
- **Les Features Extraites (features/)** : Les fichiers numpy binaires (.npy) stockant les features (train et test) générées pour chaque dataset et extracteur.
- **Le Fichier requirements.txt** : Spécifie les versions des bibliothèques logicielles (torch, scikit-learn, etc.).
- **Le Fichier README.txt** : Fichier texte simple décrivant précisément l'installation et les commandes de lancement des pipelines.
- **La Vidéo de Démonstration (Obligatoire, 2-3 minutes)** : Présente le projet.

7. RÉSULTATS EXPÉRIMENTAUX COMPLETS

Voici le tableau comparatif de l'ensemble de nos expériences (les meilleurs modèles de chaque jeu de données sont mis en valeur avec un accent de couleur vert) :

Tableau 1 : Résultats comparatifs des métriques de classification sur les 4 datasets

Dataset	Pipeline	Modèle / Config	Accuracy	F1-Score	Temps (s)	Paramètres
Iris	Pipeline 1	k-NN (via VGG16)	66.93%	56.38%	1.85s	138 357 544
Iris	Pipeline 1	SVM (via InceptionV3)	66.14%	58.78%	16.50s	27 161 264
Iris	Pipeline 2	CNN Personnalisé	49.61%	50.45%	278.74s	8 781 059
Covid19	Pipeline 1	SVM (via InceptionV3)	82.51%	82.53%	61.75s	27 161 264
Covid19	Pipeline 1	k-NN (via InceptionV3)	76.68%	76.68%	3.27s	27 161 264
Covid19	Pipeline 2	CNN Personnalisé	60.09%	54.15%	496.86s	8 780 546
Wildfire	Pipeline 1	k-NN (via VGG16)	99.45%	99.45%	26.56s	138 357 544
Wildfire	Pipeline 1	SVM (via AlexNet)	99.27%	99.27%	345.63s	61 100 840
Wildfire	Pipeline 2	CNN Personnalisé	98.00%	98.00%	1012.78s	8 780 546
DTD	Pipeline 1	SVM (via InceptionV3)	98.77%	98.76%	14.10s	27 161 264
DTD	Pipeline 1	k-NN (via InceptionV3)	98.77%	98.76%	1.35s	27 161 264
DTD	Pipeline 2	CNN Personnalisé	59.88%	60.30%	375.75s	8 784 137

8. ANALYSE ET INTERPRÉTATION PAR JEU DE DONNÉES

8.1. Jeu de données Iris Flowers

Le dataset Iris possède un faible nombre d'images (421 images, 3 classes). Le k-NN appliqué aux features de VGG16 obtient le meilleur score avec 66.93% de précision globale. Le classifieur profite des descripteurs puissants de VGG16 qui est déjà capable de distinguer les formes fines de pétales. Notre CNN personnalisé entraîné à partir de zéro stagne à 49.61%. Cela démontre que sans une base de données de taille conséquente, un CNN from scratch peine à converger vers des descripteurs robustes.

Figure 1 : Courbe d'apprentissage du CNN personnalisé sur Iris

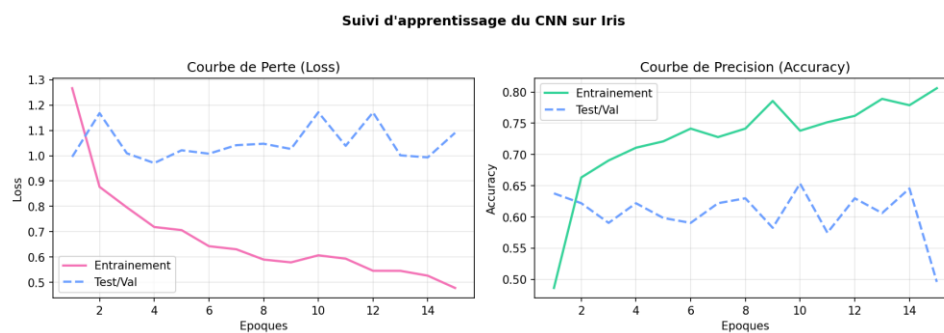
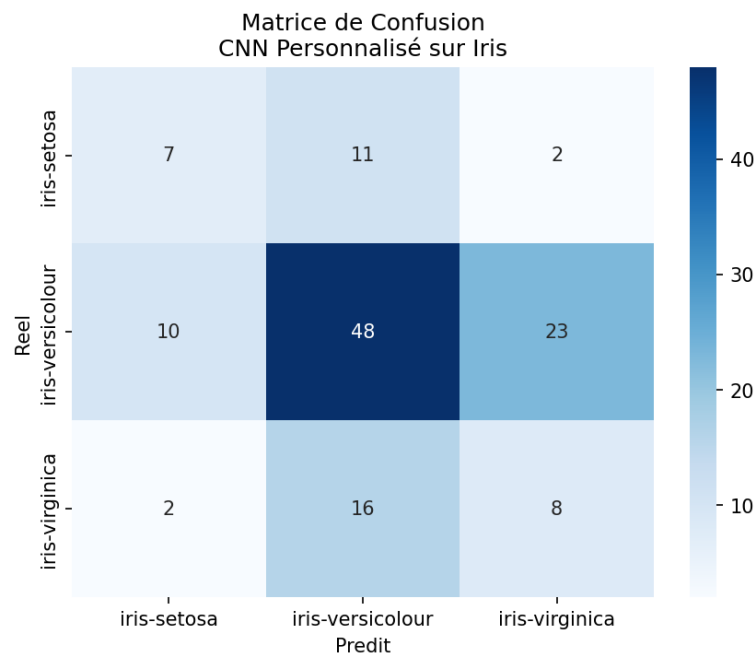


Figure 2 : Matrice de confusion du CNN personnalisé sur Iris



8.2. Jeu de données COVID-19 X-Ray

Sur ce dataset de 743 radiographies thoraciques, le SVM combiné aux caractéristiques d'InceptionV3 produit le meilleur résultat avec 82.51% d'accuracy globale. Le CNN personnalisé atteint quant à lui 60.09%. Les structures visuelles de ces radiographies pulmonaires sont subtiles et nécessitent le pouvoir d'analyse multi-échelle d'InceptionV3.

Figure 3 : Matrice de confusion SVM + InceptionV3 sur Covid19

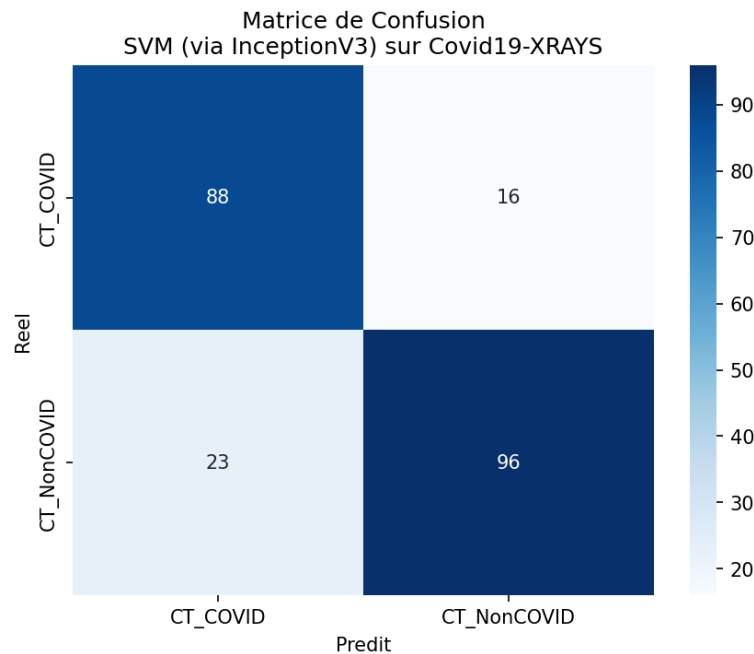
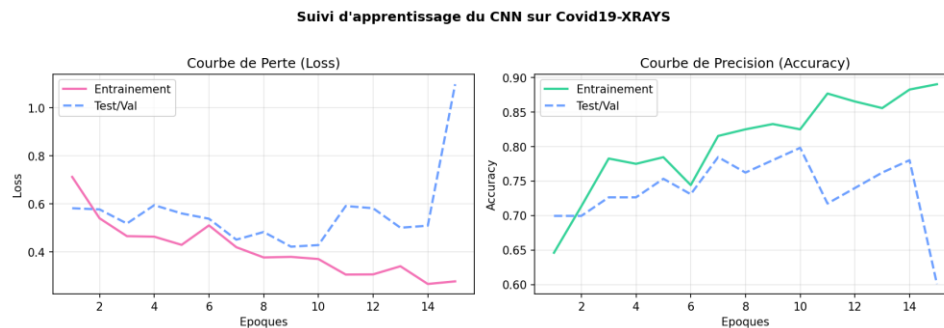


Figure 4 : Courbes d'apprentissage du CNN personnalisé sur Covid19



8.3. Jeu de données Wildfire Satellite Images

Les images satellites présentent des contrastes très marqués. De ce fait, tous nos modèles obtiennent des résultats spectaculaires. Le k-NN couplé à VGG16 obtient 99.45% d'accuracy. Le SVM sur AlexNet le talonne à 99.27%. Plus intéressant encore, notre CNN personnalisé atteint un score remarquable de 98.00%. C'est sur ce dataset, qui est le plus grand (1 832 images), que notre CNN exprime son plein potentiel.

Figure 5 : Courbe ROC-AUC du k-NN sur VGG16 (Wildfire)

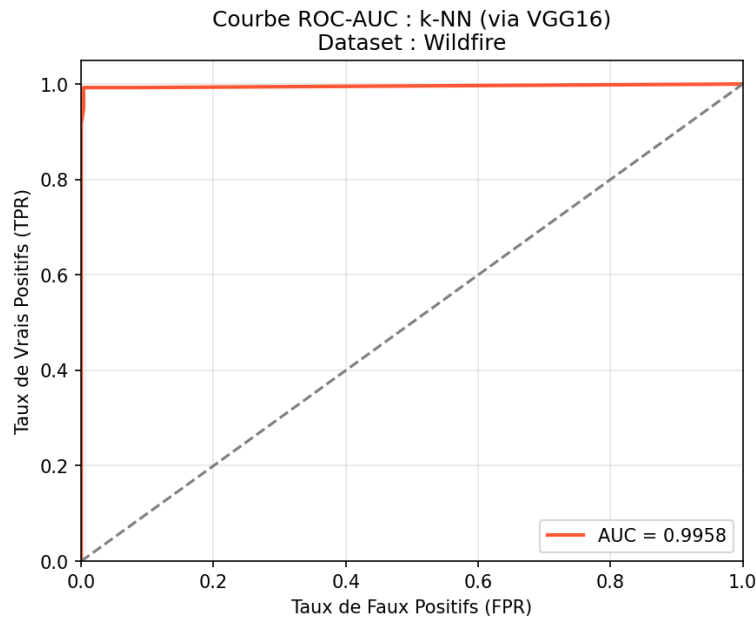
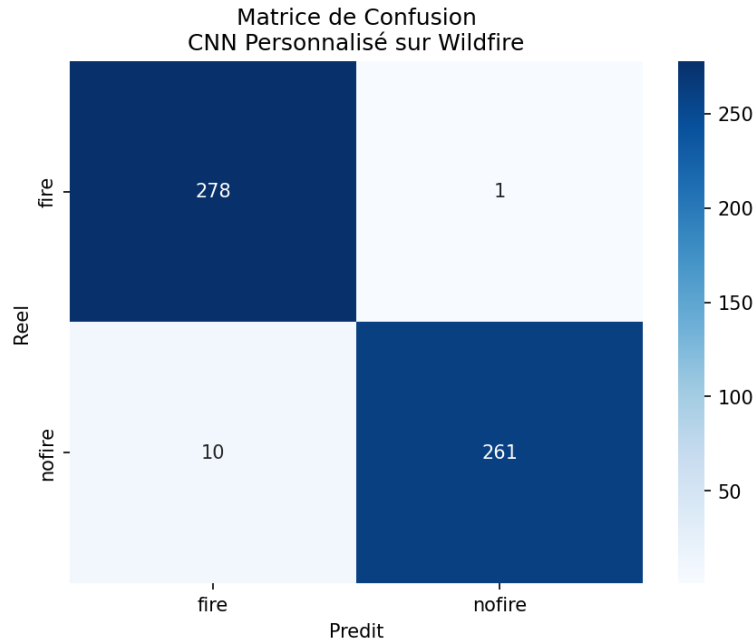


Figure 6 : Matrice de confusion du CNN personnalisé (Wildfire)



8.4. Jeu de données DTD (Textures complexes)

Le dataset DTD propose 9 classes d'animaux pour seulement 540 images au total. La difficulté est extrême pour notre CNN personnalisé qui stagne à 59.88% d'accuracy globale. À l'inverse, la puissance d'extraction d'InceptionV3 couplée à un SVM ou à un k-NN permet de résoudre le problème presque parfaitement avec un score exceptionnel de 98.77%.

Figure 7 : Courbes ROC du SVM sur InceptionV3 (DTD)

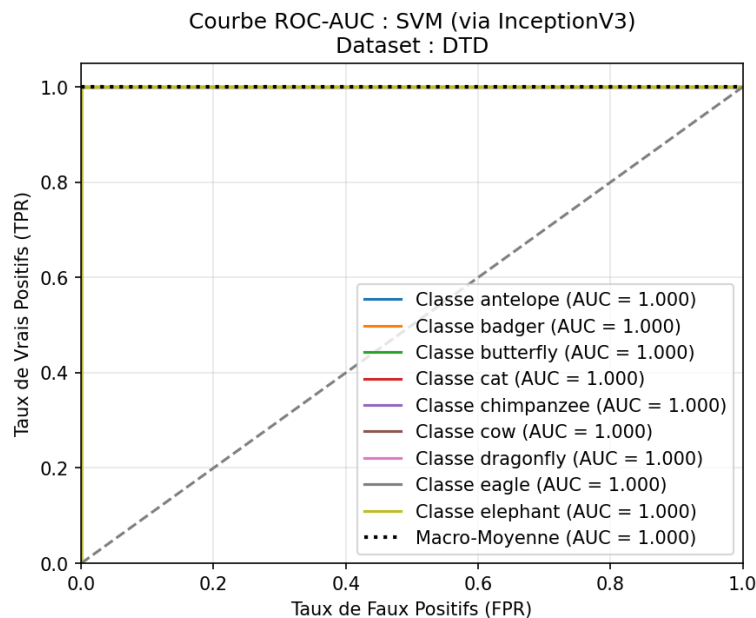
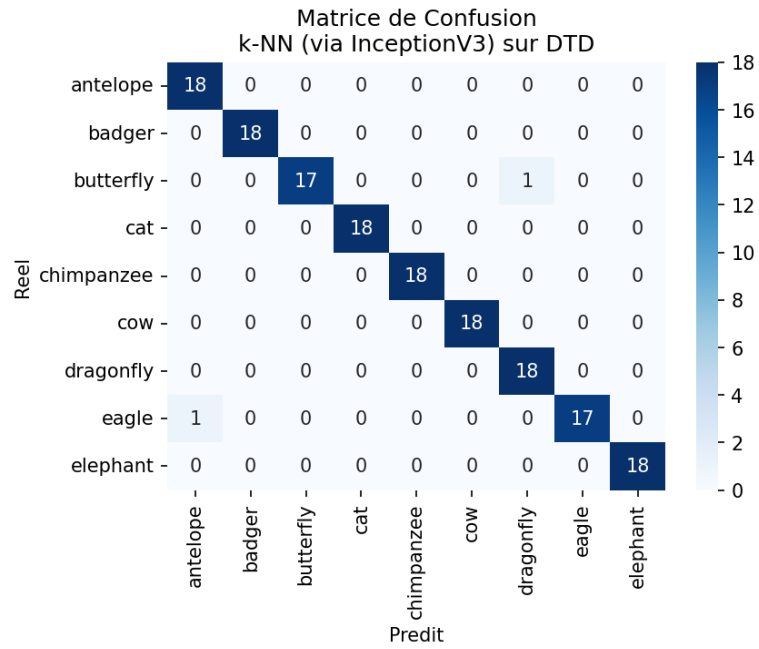


Figure 8 : Matrice de confusion du k-NN sur InceptionV3 (DTD)



9. SYNTHÈSE VISUELLE ET COMPARAISON GLOBALE

Pour résumer visuellement nos analyses, nous affichons ci-dessous la synthèse globale des performances.

Figure 9 : Comparaison des performances par Dataset (Pipeline 1 vs Pipeline 2)

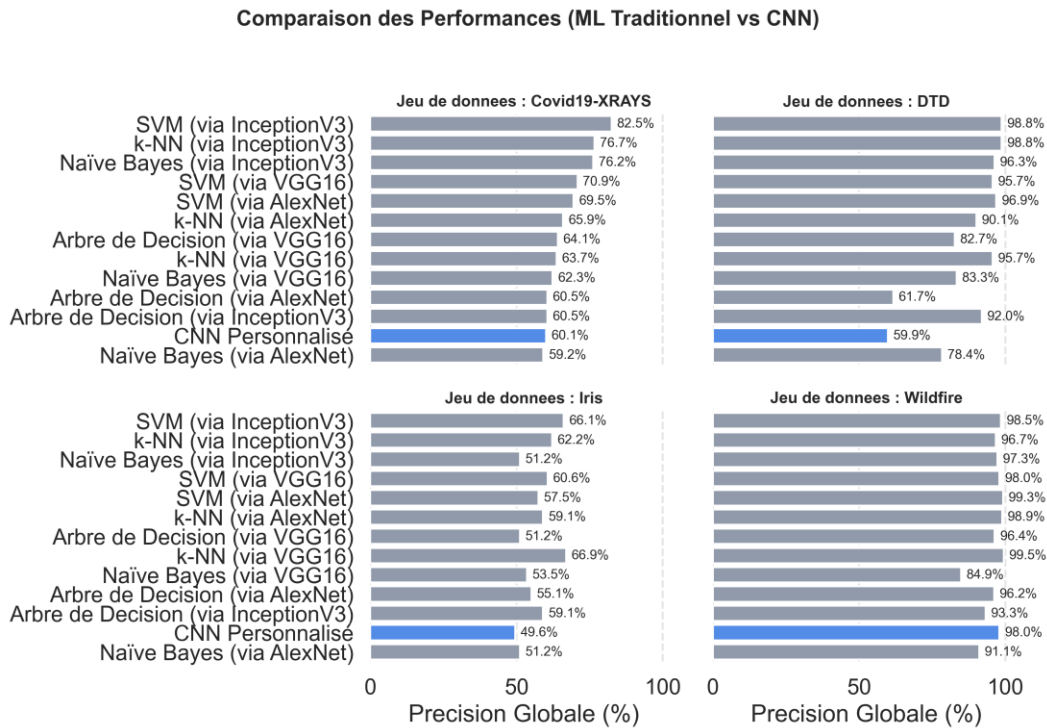
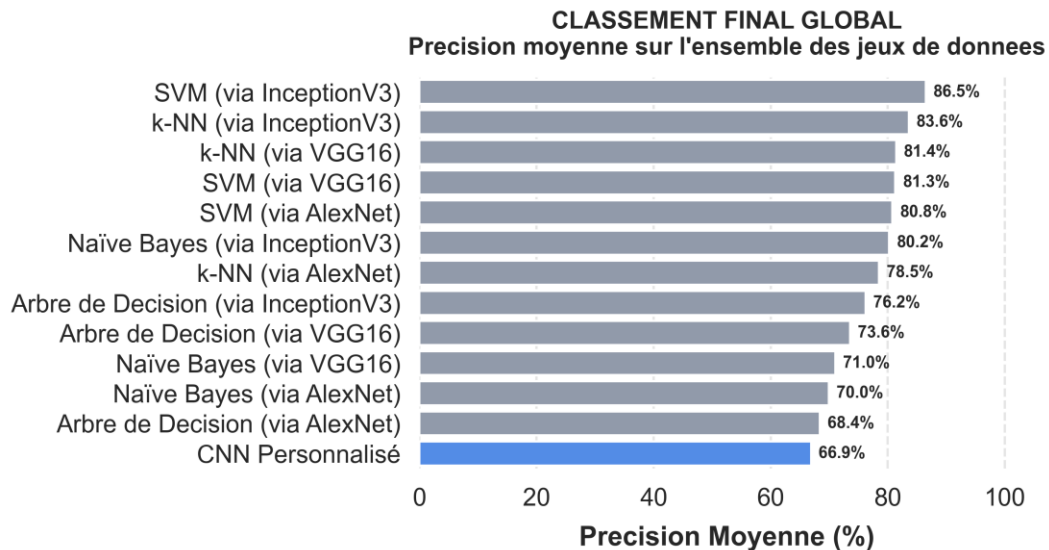


Figure 10 : Classement Final Global Moyen sur tous les Datasets



10. DISCUSSION CRITIQUE ET ANALYSE DES COMPROMIS

- **Efficacité du Transfer Learning (Pipeline 1) :** Le Transfer Learning s'impose comme le grand vainqueur de cette étude de manière systématique. Son avantage est flagrant sur les petits jeux de données (Iris, DTD) où un apprentissage from scratch souffre prématurément d'un sévère surapprentissage. Le fait d'utiliser des extracteurs de caractéristiques pré-entraînés sur ImageNet permet de bénéficier d'un espace de représentation visuel déjà universellement optimisé.
- **Coût d'Apprentissage et Temps d'Entraînement :** Les modèles traditionnels (SVM, k-NN) sur descripteurs pré-extraits s'entraînent en une fraction de seconde (de 0.3 à 30 secondes). En revanche, la phase initiale d'extraction représente le goulot d'étranglement. Pour le CNN du Pipeline 2, le temps d'entraînement est extrêmement élevé (plus de 16 minutes sur CPU pour 15 époques sur le dataset Wildfire).
- **Taille des Modèles et Déploiement Embarqué :** Bien que le Pipeline 1 soit performant, il impose une contrainte mémoire majeure. VGG16 possède 138 millions de paramètres (> 500 Mo sur le disque), alors que notre CNN personnalisé ne nécessite que 8.7 millions de paramètres (environ 35 Mo). Ce modèle est donc de 3 à 15 fois plus compact que les modèles de transfert, ce qui en fait un compromis extrêmement attractif pour de l'informatique embarquée.

CONCLUSION ET RECOMMANDATIONS

En réponse aux problématiques établies, nous proposons les recommandations d'usage suivantes:

- **Volume de données réduit (< 1000 images) :** Il faut obligatoirement privilégier le Pipeline 1 (InceptionV3 ou VGG16 associés à un classifieur SVM ou k-NN) pour obtenir une précision élevée en évitant le surapprentissage.
- **Volume de données important (> 5000 images) :** Le Pipeline 2 (CNN personnalisé) est à considérer, surtout si la taille de stockage finale et l'empreinte mémoire du modèle sont des contraintes de déploiement critiques.
- **Téléchargement et ressources de calcul limitées:** Les classifieurs classiques couplés au cache d'extraction permettent d'itérer et de tester des modèles en quelques secondes sur de simples architectures CPU.

LISTE DES FIGURES

- **Figure 1** : Courbe d'apprentissage du CNN personnalisé sur Iris
- **Figure 2** : Matrice de confusion du CNN personnalisé sur Iris
- **Figure 3** : Matrice de confusion SVM + InceptionV3 sur Covid19
- **Figure 4** : Courbes d'apprentissage du CNN personnalisé sur Covid19
- **Figure 5** : Courbe ROC-AUC du k-NN sur VGG16 (Wildfire)
- **Figure 6** : Matrice de confusion du CNN personnalisé (Wildfire)
- **Figure 7** : Courbes ROC du SVM sur InceptionV3 (DTD)
- **Figure 8** : Matrice de confusion du k-NN sur InceptionV3 (DTD)
- **Figure 9** : Comparaison des performances par Dataset (Pipeline 1 vs Pipeline 2)
- **Figure 10** : Classement Final Global Moyen sur tous les Datasets